Day 6 Evidence Workbooklet — Instructor Key

Branch Outcomes with if and else

Engineering practice → condition result → branch followed → selected action → support plan

Instructor key. Same layout as student copy; solutions filled in response boxes. Print format: duplex, left-stapled packet.

Name		**Section**		**Date**	
Score	_____/42		Mastery check	secure / developing / needs support	

The Week 2 scenario starts with a sensor-kit checkout table. A kit may be released, charged, sent to calibration, or held for missing checkout information. Today the focus is a single branch decision: read one condition, choose the branch that runs, and explain the action selected.

Engineering task	Decide which checkout action a sensor kit should receive from one condition at a time.
Computational move	Turn a true/false condition into one selected branch outcome while keeping the evidence for that branch outcome visible.
Python focus	`if`, `else`, condition result, selected branch, action variable, and printed branch outcome evidence.
Evidence to keep	case value, condition expression, `True`/`False` result, branch followed, action assigned, and support note.

Day 6 working rules

1. Python checks the condition after `if`.
2. If the condition is `True`, the `if` branch runs.
3. If the condition is `False`, the `else` branch runs.
4. Only one of those two branches runs.
5. A reliable branch note names both the condition result and the selected action.

1 Read the checkout condition before choosing an action

recognize predict

The lab team is checking whether a sensor kit can leave the checkout table. Day 6 uses one decision at a time: if the condition is true, one action happens; otherwise the other action happens.

```
kit_label = "S-14"
charge_percent = 82
charge_minimum = 80
calibration_label = "current"
borrower_initials = "JL"

charge_ok = charge_percent >= charge_minimum
calibration_ok = calibration_label == "current"
borrower_logged = borrower_initials != ""

if charge_ok:
    action = "send to calibration check"
else:
    action = "charge before checkout"
```

Pts.	Prompt	My evidence
1	Which variable stores the kit label?	**Key:** `kit_label`.
1	Which comparison creates `charge_ok`?	**Key:** `charge_percent >= charge_minimum`; here that is `82 >= 80`.
1	Is `charge_ok` true or false for this kit? Show the numbers.	**Key:** `True`; `82 >= 80`.
1	Which branch assigns the action for this kit?	**Key:** The `if` branch runs because `charge_ok` is `True`; it assigns `send to calibration check`.

2 Trace an `if/else` branch outcome

trace explain

Complete the branch outcome table. Use the condition result to decide which branch runs.

Pts.	Charge value	Condition	Branch followed	Action and reason
2	82	`charge_percent >= 80`	**Key:** see reason	**Key:** Result `True`; follow the `if` branch and set action to `send to calibration check` because `82 >= 80`.
2	80	`charge_percent >= 80`	**Key:** see reason	**Key:** Result `True`; follow the `if` branch and set action to `send to calibration check`; equality is included by `>=`.
2	79	`charge_percent >= 80`	**Key:** see reason	**Key:** Result `False`; follow the `else` branch and set action to `charge before checkout` because `79 < 80`.
2	Explain why the exact value `80` matters.	**Key:** `80` is the boundary. With `>=`, `80 >= 80` is `True`, so the exact minimum follows the `if` branch.		

Boundary connection

The branch outcome depends on a comparison from Week 1. If the comparison is wrong, the selected branch outcome will be wrong too.

3 Choose the selected action from printed evidence

predict explain

The same structure is used for calibration status. Read the code and predict the action.

```
calibration_label = "expired"
calibration_ok = calibration_label == "current"

if calibration_ok:
    checkout_status = "ready for borrower log"
else:
    checkout_status = "send to calibration bench"

print(kit_label, checkout_status)
```

Pts.	Prompt	My evidence
2	What value is stored in `calibration_ok`? Why?	**Key:** `False`, because `calibration_label` is `expired`, not `current`.
2	Which branch runs: the `if` branch or the `else` branch?	**Key:** The `else` branch runs because `calibration_ok` is `False`.
2	What exact status text is printed after the kit label?	**Key:** `send to calibration bench`.
2	Why would `bool(calibration_label)` be unsafe here?	**Key:** Any nonempty label such as `expired` is truthy, so it could look acceptable even though the needed value is exactly `current`.

4 Complete one branch condition

implement debug

Complete the condition so the kit moves to borrower logging only when the borrower initials are present.

```
if ________________________________________________:
    next_step = "record borrower and release kit"
else:
    next_step = "hold until borrower initials are entered"
```

Pts.	Prompt	My response
3	Write the completed condition using `borrower_initials`.	**Key:** `borrower_initials != ""`.
2	Why should the condition compare against an empty string instead of checking the kit label?	**Key:** The branch is about whether borrower initials were entered. A kit label can exist even when `borrower_initials` is empty.
2	Give one input value that should follow the `else` branch.	**Key:** `borrower_initials = ""` should follow the `else` branch.
1	Name the evidence type you used: state / type / boundary / branch.	**Key:** branch.

5 Debug a branch-outcome claim

debug test explain

A teammate writes: "The kit is ready because `charge_ok` is true." Use branch-outcome evidence to evaluate that claim.

Pts.	Prompt	My response
2	What part of the claim is supported by the code?	**Key:** The charge evidence is supported: `charge_ok` is `True` because `82 >= 80`.
2	What evidence is still missing before a full checkout release?	**Key:** Need calibration and borrower evidence, such as `calibration_ok` being `True` and `borrower_initials` not being empty.
2	Give one test case that should follow the hold or repair outcome.	**Key:** Example: `charge_percent = 82`, `calibration_label = "expired"`, and initials present should produce calibration support, not release.
2	Write a safer one-sentence checkout note.	**Key:** The kit has enough charge, but release should wait until calibration is `current` and borrower initials are present; otherwise use the matching support plan.

6 Transfer and support plan

transfer explain

Pts.	My checkout-decision note
4	Write a short note that says how an `if/else` statement chooses one action from one condition. Use the sensor-kit checkout example. **Key:** An `if/else` statement checks one condition and runs exactly one branch. For charge, `82 >= 80` is `True`, so the `if` branch chooses `send to calibration check`; a low charge would choose `charge before checkout`.
2	Circle the support plan you would use next: condition result / branch followed / exact boundary / string label / written explanation. Name one next action: _____

Bridge to Day 7

Day 6 chooses between two actions. Day 7 adds ordered rules where the first true condition wins and later branches are skipped.

VIP Lab Alignment

Bridge Day 6 • Contract 7cc3f8d502f3

This page replaces the earlier wrapper-based lab bridge and follows the current top-level notebook interface.

Why this page is here

VIP Lab Day(s): 3 **Activities:** 18.8, 18.9, 18.10, 18.11, 18.12

Finish the current calculation-and-output lab with top-level code cells; do not use or author a function wrapper.

Open these current notebook cells

Activity / stable cells	Notebook work	Evidence to carry forward
18.8 18-8-work / 18-8-transfer	**Badge Sheet Decision**	Calculate a total from two named quantities.
18.9 18-9-work / 18-9-transfer	**Simple Statistics**	Use named results for minimum, maximum, total, and average.
18.10 18-10-work / 18-10-transfer	**Box Model Values**	Translate the volume and surface-area formulas into named calculations.
18.11 18-11-work / 18-11-transfer	**Structured Text Shape**	Treat characters and line order as exact output requirements.
18.12 18-12-work / 18-12-transfer	**Storage Bay Estimate**	Translate a rectangular-volume estimate into one expression.

Readiness check before you code

1. For Activity 18.8, write the two exact boundary values that must be tested before you open the notebook.

   Answer and explanation: The exact decision boundaries are 8 and 16. Test values immediately below, at, and immediately above each boundary.

2. Choose one Activity 18.9-18.12 and name the intermediate value and exact displayed result you will inspect first.

   Answer and explanation: Accept any activity-specific answer that names a real intermediate quantity and its required output, such as total and average for 18.9 or volume and surface area for 18.10.

Notebook and submission procedure

1. Use the sample and transfer cells for formative practice; attempt before opening a reveal.
2. Complete the end-of-notebook cold cells without a reveal or copied solution. Only those cold cells are scored for independent mastery on Lab Days 5–7.
3. Save or download the completed notebook as one `.ipynb` file.
4. Upload that one notebook to the matching Gradescope assignment. Do not export a `.py` file or create a ZIP.